

A generic back-end for exploratory programming

Damian Frölich^{1,2} and Thomas van Binsbergen¹

¹University of Amsterdam

²VU Amsterdam

February 2021



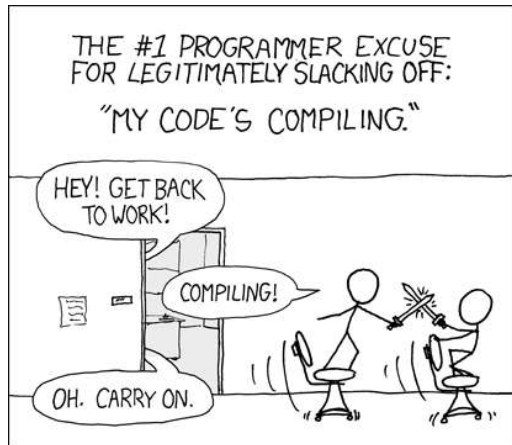
UNIVERSITY
OF AMSTERDAM



VRIJE
UNIVERSITEIT
AMSTERDAM

Programming forms

- ▶ Edit → Compile → Run
 - ▶ Slow interactivity
 - ▶ Shallow interactivity



<https://xkcd.com/303/>

REPL

- ▶ Incremental programming
- ▶ Immediate feedback
- ▶ Provides some form of exploratory programming

```
In [1]: def hello():  
...:     print("Hello")  
...:  
  
In [2]: hello()  
Hello  
  
In [3]: def hello():  
...:     print("World")  
...:  
...:  
  
In [4]: hello()  
World
```

Notebook

- ▶ Alternative interface
- ▶ Combines with literate programming

Welcome to Jupyter!

```
In [1]: print("Hello world")
```

```
Hello world
```

Inconsistent interfaces

```
jshell> int x;  
x ==> 0  
jshell>  
    class A {  
        public void run() {  
            x++;  
        }  
    }  
| created class A  
jshell> A a = new A();  
a ==> A@5ce65a89  
jshell> a.run()  
jshell> x  
x ==> 1
```

This is a *markdown* cell

In [1]: `int x;`

In [2]: `class A { public void run() { x++; } }`

In [3]: `A a = new A();`

In [4]: `a.run();`

Only the cell below produces output

In [5]: `x`

Out[5]: 1

Extension of a language

- ▶ (some) Interfaces require an extension on the original language
 - ▶ Not always documented
 - ▶ Independent of original language

Principled approach¹

- ▶ Definitional interpreter
 - ▶ Difference between base and extension language
 - ▶ Generic interfaces
 - ▶ **Exploratory programming**

```
Config eval((Phrase)`<Expression e> `, Config c)
  = catchExceptions(collectBindings(
    setOutput(createBinding(eval(c, e)))));

Config eval((Phrase)`<Statement s>`, Config c)
  = catchExceptions(collectBindings(
    setOutput(exec(s, c))));

Config eval((Phrase)`<ClassDecl cd>`, Config c)
  = catchExceptions(collectBindings(
    declareClass(cd, c)));

Config eval((Phrase)`<VarDecl vd>`, Config c)
  = catchExceptions(collectBindings(
    declareVariables(vd, c)));

Config eval((Phrase)`<MethodDecl md>`, Config c)
  = catchExceptions(collectBindings(
    declareGlobalMethod(md, c)));

Config eval((Phrase)`<Phrase p1> <Phrase p2>`, Config c)
  = eval(p2, eval(p1, c));
```

¹Binsbergen et al. 2020

Exploratory programming

- ▶ Exploring interpreter
 - ▶ Current configuration
 - ▶ Execution graph
 - ▶ Operations
 - ▶ Display
 - ▶ Execute
 - ▶ Revert

Exploratory programming

- ▶ Exploring interpreter
 - ▶ Current configuration
 - ▶ Execution graph
 - ▶ Operations
 - ▶ Display
 - ▶ Execute
 - ▶ Revert
- ▶ Execution graph behaviour
 - ▶ Stack
 - ▶ Tree
 - ▶ Graph
 - ▶ Graph-structured Stack(GSS)

Behaviour examples

Tree behaviour

Graph behaviour

Definition

A language L is a structure $\langle P, \Gamma, \gamma^0, I \rangle$ with:

- P a set of programs,
- Γ a set of configurations,
- $\gamma^0 \in \Gamma$ an initial configuration and
- I a definitional interpreter assigning to each program $p \in P$ a function $I_p : \Gamma \rightarrow \Gamma$.

Definition

A language L is a structure $\langle P, \Gamma, \gamma^0, I \rangle$ with:

P a set of programs,

Γ a set of configurations,

$\gamma^0 \in \Gamma$ an initial configuration and

I a definitional interpreter assigning to each program $p \in P$ a function $I_p : \Gamma \rightarrow \Gamma$.

```
whileInterpreter :: Command -> Config -> Config
```

```
data Config = Config { cfgStore :: Store, cfgOutput :: Output }
```

```
type Store = Map String Literal
```

```
type Output = [String]
```

```
initialConfig = Config { cfgStore = empty, cfgOutput = [] }
```

Definition

A language $L = \langle P, \Gamma, \gamma^0, I \rangle$ is *sequential* if there is an operator $\otimes$ such that for every $p_1, p_2 \in P$ and $\gamma \in \Gamma$ it holds that $p_1 \otimes p_2 \in P$ and that $I_{p_1 \otimes p_2}(\gamma) = (I_{p_2} \circ I_{p_1})(\gamma)$.

Definition

A language $L = \langle P, \Gamma, \gamma^0, I \rangle$ is *sequential* if there is an operator $\otimes$ such that for every $p_1, p_2 \in P$ and $\gamma \in \Gamma$ it holds that $p_1 \otimes p_2 \in P$ and that $I_{p_1 \otimes p_2}(\gamma) = (I_{p_2} \circ I_{p_1})(\gamma)$.

While is sequential

```
whileInterpreter (Seq p_1 p_2) gamma ==  
  (whileInterpreter p_2 . whileInterpreter p_1) gamma
```

Contribution

Provide a generic exploring interpreter allowing experimentation with the different execution graph behaviours for different type of languages and interfaces.

Exploring interpreter

```
data Explorer programs configs = Explorer
{ defInterp :: programs -> configs -> configs
, config :: configs
, execEnv :: Gr _ programs
}
```



```
type Ref = Int
data Explorer programs configs = Explorer
{ ...
  , execEnv :: Gr Ref programs
  , currRef :: Ref
  , cmap    :: Map Ref configs
}
```

```
data Explorer programs configs = Explorer
{ ...
, sharing :: Bool
, backtracking :: Bool
}
```

	Sharing	No sharing
No backtracking	Graph	Tree
Backtracking	GSS	Stack

Full definition

```
data Explorer programs configs = Explorer
{ defInterp :: programs -> configs -> configs
, config    :: configs
, execEnv   :: Gr Ref programs
, currRef   :: Ref
, cmap      :: Map Ref configs
, sharing   :: Bool
, backtracking :: Bool
}
```

```
mkExplorerStack :: (a -> b -> b) -> b -> Explorer a b  
mkExplorerStack = mkExplorer False True
```

```
mkExplorerTree  :: (a -> b -> b) -> b -> Explorer a b  
mkExplorerTree  = mkExplorer False False
```

```
mkExplorerGraph :: (a -> b -> b) -> b -> Explorer a b  
mkExplorerGraph = mkExplorer True False
```

```
mkExplorerGSS :: (a -> b -> b) -> b -> Explorer a b  
mkExplorerGSS = mkExplorer True True
```

Operations

```
revert :: Ref -> Explorer p c -> Maybe (Explorer p c)  
execute :: (Eq c, Eq p) => p -> Explorer p c -> Explorer p c
```

Unpacking

Sequence of programs results in one transition

```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
      ~~~~~
```



```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
      ~~~~~
```

```
executeAll :: (Eq c, Eq p) => [p] -> Explorer p c -> Explorer p c
executeAll = flip (foldr execute)
```

Future work

Side effects?

- ▶ `execute :: Monad m => p -> Explorer p c -> m (Explorer p c)`

Debugging

Interface exploration

- ▶ Different type of interfaces
- ▶ Integration with different exploration behaviours

Evaluation with eFLINT

frames

```
1. pay: buyer
2. pay: seller
3. pay: buyer
4. pay: seller
5. pay: buyer
6. pay: seller
7. pay: buyer
8. pay: seller
9. pay: buyer
10. pay: seller
11. pay: buyer
12. pay: seller
13. pay: buyer
14. pay: seller
15. pay: buyer
16. pay: seller
17. pay: buyer
18. pay: seller
19. pay: buyer
20. pay: seller
21. pay: buyer
22. pay: seller
23. pay: buyer
24. pay: seller
25. pay: buyer
26. pay: seller
27. pay: buyer
28. pay: seller
29. pay: buyer
30. pay: seller
31. pay: buyer
32. pay: seller
33. pay: buyer
34. pay: seller
35. pay: buyer
36. pay: seller
37. pay: buyer
38. pay: seller
39. pay: buyer
40. pay: seller
41. pay: buyer
42. pay: seller
43. pay: buyer
44. pay: seller
45. pay: buyer
46. pay: seller
47. pay: buyer
48. pay: seller
49. pay: buyer
50. pay: seller
51. pay: buyer
52. pay: seller
53. pay: buyer
54. pay: seller
55. pay: buyer
56. pay: seller
57. pay: buyer
58. pay: seller
59. pay: buyer
60. pay: seller
61. pay: buyer
62. pay: seller
63. pay: buyer
64. pay: seller
65. pay: buyer
66. pay: seller
67. pay: buyer
68. pay: seller
69. pay: buyer
70. pay: seller
71. pay: buyer
72. pay: seller
73. pay: buyer
74. pay: seller
75. pay: buyer
76. pay: seller
77. pay: buyer
78. pay: seller
79. pay: buyer
80. pay: seller
81. pay: buyer
82. pay: seller
83. pay: buyer
84. pay: seller
85. pay: buyer
86. pay: seller
87. pay: buyer
88. pay: seller
89. pay: buyer
90. pay: seller
91. pay: buyer
92. pay: seller
93. pay: buyer
94. pay: seller
95. pay: buyer
96. pay: seller
97. pay: buyer
98. pay: seller
99. pay: buyer
100. pay: seller
```

scenario

```
1. pay: buyer
2. pay: seller
3. pay: buyer
4. pay: seller
5. pay: buyer
6. pay: seller
7. pay: buyer
8. pay: seller
9. pay: buyer
10. pay: seller
11. pay: buyer
12. pay: seller
13. pay: buyer
14. pay: seller
15. pay: buyer
16. pay: seller
17. pay: buyer
18. pay: seller
19. pay: buyer
20. pay: seller
21. pay: buyer
22. pay: seller
23. pay: buyer
24. pay: seller
25. pay: buyer
26. pay: seller
27. pay: buyer
28. pay: seller
29. pay: buyer
30. pay: seller
31. pay: buyer
32. pay: seller
33. pay: buyer
34. pay: seller
35. pay: buyer
36. pay: seller
37. pay: buyer
38. pay: seller
39. pay: buyer
40. pay: seller
41. pay: buyer
42. pay: seller
43. pay: buyer
44. pay: seller
45. pay: buyer
46. pay: seller
47. pay: buyer
48. pay: seller
49. pay: buyer
50. pay: seller
51. pay: buyer
52. pay: seller
53. pay: buyer
54. pay: seller
55. pay: buyer
56. pay: seller
57. pay: buyer
58. pay: seller
59. pay: buyer
60. pay: seller
61. pay: buyer
62. pay: seller
63. pay: buyer
64. pay: seller
65. pay: buyer
66. pay: seller
67. pay: buyer
68. pay: seller
69. pay: buyer
70. pay: seller
71. pay: buyer
72. pay: seller
73. pay: buyer
74. pay: seller
75. pay: buyer
76. pay: seller
77. pay: buyer
78. pay: seller
79. pay: buyer
80. pay: seller
81. pay: buyer
82. pay: seller
83. pay: buyer
84. pay: seller
85. pay: buyer
86. pay: seller
87. pay: buyer
88. pay: seller
89. pay: buyer
90. pay: seller
91. pay: buyer
92. pay: seller
93. pay: buyer
94. pay: seller
95. pay: buyer
96. pay: seller
97. pay: buyer
98. pay: seller
99. pay: buyer
100. pay: seller
```

response

ok

output

```
1. pay: buyer
2. pay: seller
3. pay: buyer
4. pay: seller
5. pay: buyer
6. pay: seller
7. pay: buyer
8. pay: seller
9. pay: buyer
10. pay: seller
11. pay: buyer
12. pay: seller
13. pay: buyer
14. pay: seller
15. pay: buyer
16. pay: seller
17. pay: buyer
18. pay: seller
19. pay: buyer
20. pay: seller
21. pay: buyer
22. pay: seller
23. pay: buyer
24. pay: seller
25. pay: buyer
26. pay: seller
27. pay: buyer
28. pay: seller
29. pay: buyer
30. pay: seller
31. pay: buyer
32. pay: seller
33. pay: buyer
34. pay: seller
35. pay: buyer
36. pay: seller
37. pay: buyer
38. pay: seller
39. pay: buyer
40. pay: seller
41. pay: buyer
42. pay: seller
43. pay: buyer
44. pay: seller
45. pay: buyer
46. pay: seller
47. pay: buyer
48. pay: seller
49. pay: buyer
50. pay: seller
51. pay: buyer
52. pay: seller
53. pay: buyer
54. pay: seller
55. pay: buyer
56. pay: seller
57. pay: buyer
58. pay: seller
59. pay: buyer
60. pay: seller
61. pay: buyer
62. pay: seller
63. pay: buyer
64. pay: seller
65. pay: buyer
66. pay: seller
67. pay: buyer
68. pay: seller
69. pay: buyer
70. pay: seller
71. pay: buyer
72. pay: seller
73. pay: buyer
74. pay: seller
75. pay: buyer
76. pay: seller
77. pay: buyer
78. pay: seller
79. pay: buyer
80. pay: seller
81. pay: buyer
82. pay: seller
83. pay: buyer
84. pay: seller
85. pay: buyer
86. pay: seller
87. pay: buyer
88. pay: seller
89. pay: buyer
90. pay: seller
91. pay: buyer
92. pay: seller
93. pay: buyer
94. pay: seller
95. pay: buyer
96. pay: seller
97. pay: buyer
98. pay: seller
99. pay: buyer
100. pay: seller
```

```
seller("Alice")
buyer("Bob")
duty to deliver(seller("Alice"),buyer("Bob"))
duty to pay(buyer("Bob"),seller("Alice"))
suspend(10)
pay(buyer("Bob"),seller("Alice"),amount(10))
amount(10)
suspend(seller("Alice"),buyer("Bob"),amount(10))
query successful
query successful
P1 = 10
actions & events:
1. deliver(seller("Alice"),buyer("Bob"),amount(10)) (ENABLED)
2. pay(buyer("Bob"),seller("Alice"),amount(10)) (ENABLED)
3. suspend-delivery(seller("Alice"),buyer("Bob")) (DISABLED)
4. tick() (PASSING)
P1 = 10
-tick(10)
-tick(1)
P1 = 10
-tick(1)
-tick(2)
suspend-delivery(seller("Alice"),buyer("Bob"))
P1 = suspend-delivery(seller("Alice"),buyer("Bob"))
no actions
1. suspend-delivery(seller("Alice"),buyer("Bob"))
P2 = suspend-delivery(seller("Alice"),buyer("Bob"))
P2 = suspend-delivery(seller("Alice"),buyer("Bob"))
not a compliant action
P2 = 1
```

A generic back-end for exploratory programming

The implementation opens up exploration of the exploring interpreter in a language independent manner

Damian Frolich
University of Amsterdam and VU Amsterdam
d.frolich@uva.nl



Thomas van Binsbergen
University of Amsterdam
ltvanbinsbergen@acm.org

